

1. Launch the framework using the secure bash wrapper:
`./start.sh`

Core Command Modules

SuperSploit's commands are divided into highly modular registries:

General Commands

- `help` - Display the help menu.
- `show` - Show framework status and options.
- `set` - Configure dynamic framework variables (e.g., target IP, payload, ports).
- `exploit` - Compile and/or execute the currently loaded exploit.
- `search` - Search the database for specific exploits or payloads.
- `add` - Add new dynamic variables to the runtime database.

Reconnaissance Commands

- `recon-ng` - Launch the integrated Recon-ng OSINT framework.
- `name-search` - Launch the social media Google Dork generator.
- `bettercap` - Safely verify and launch Bettercap.
- `phoneinfoga` - Scan and log phone numbers.

Network Scanning (Nmap)

- `port-scan` - Perform detailed port enumeration.
- `get-targets` - Perform a full network sweep and log live hosts.
- `view-targets` / `view-targets-v` - Display tracked network targets.
- `scan-target` - Perform a granular, custom argument scan against a selected target.
- `import-targets` - Load a previous session's target database.

Bluetooth Exploitation

- `ducky` - Launch the BlueDucky payload delivery system.
- `ranger` - Target specific, known Bluetooth devices from your loot database.

Screenshots

(See *assets* directory for UI previews)

- **Start Screen:** Auto-suggest CLI and framework banners.
- **Dynamic Variables:** Setting and viewing exploit targets.
- **Search Feature:** Quickly finding payloads with multiple search arguments.
- **Exploit Details:** Reviewing parsed payload options before execution.

""""

```
with open("README.md", "w", encoding="utf-8") as f: f.write(markdown_content)
Your Markdown file is ready
```

[file-tag: code-generated-file-0-1776689625055294402]

The README has been successfully generated based on the original framework concept and the newly refactored architecture. It includes the original disclaimers and credits while properly documenting the new features such as the dictionary command registry, secure input handling with `shlex`, memory-safe Python payload execution, and the BlueDucky Bluetooth subsystem.